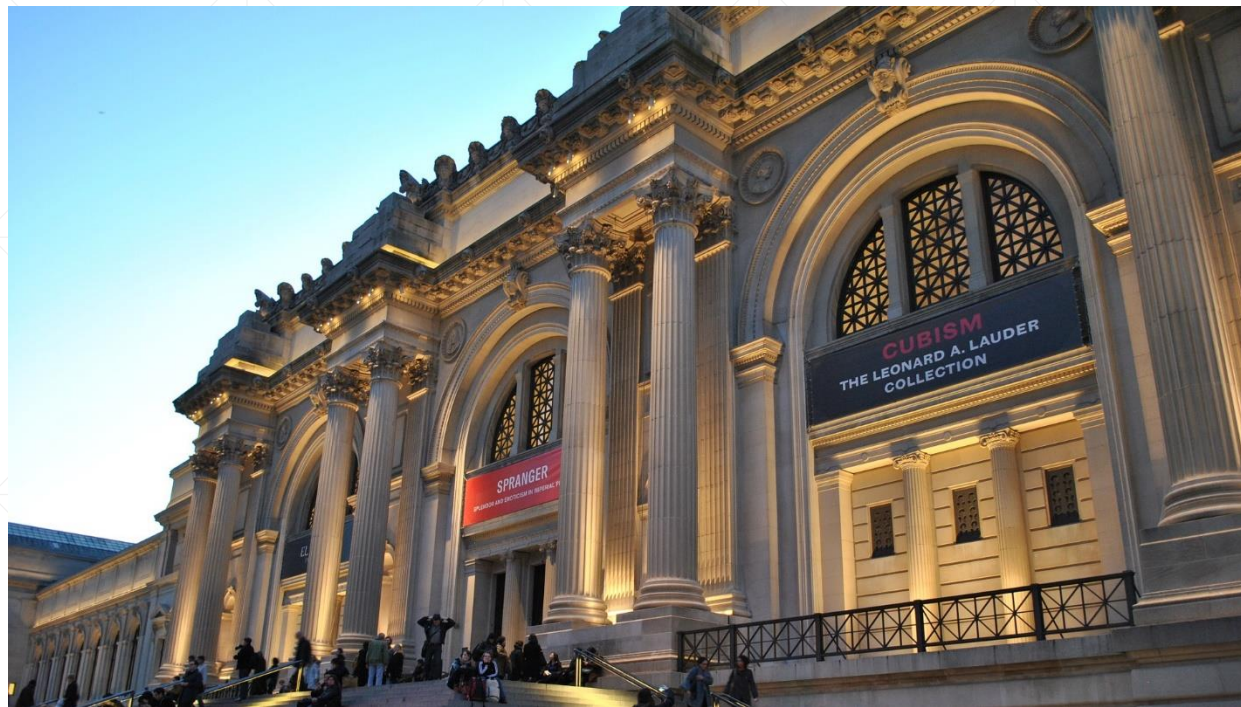




# 语句与控制结构

---

北京理工大学计算机学院  
金旭亮

# Java程序中的语句



Java程序由一条条的语句所构成，以分号结尾。



除了在字符串内部，通常情况下，语句中间可以敲回车键将其分为多行，或者使用Tab键和空格键调整其缩进，这些调整，不影响语句的编译和程序的执行。

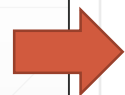


在IntelliJ IDEA中，可以使用快捷键（Ctrl + Alt + L）“自动格式化”选中的代码块。

# 语句块与变量作用域

使用“{”和“}”将多条语句包围起来，构成一个语句块。

语句块实际上是定义了一个“作用域”，在里面可以定义变量，而这个变量的生存期，仅仅只限于本语句块。



```
//语句块中变量的作用域
private static void VariableScopeInBlock() {
    int i = 1;
    //定义一个语句块
    {
        //语句块内定义的j和sum,只能在本语句块内所使用
        int j = 2;
        //在本语句块中,可以使用外部定义的变量i
        int sum = i + j;
        System.out.println(sum);//3
    }
    //在语句块外部,不能再访问变量j和sum
    //System.out.println(j);
}
```

# 语句块出现在很多地方.....

```
public class StatementBlock {
```

```
    static {
```

```
        System.out.println("类初始化语句块");
```

```
    }
```

```
    //方法（或函数）的语句块
```

```
    public static void main(String[] args) {
```

```
        //for和if语句的语句块
```

```
        for (int i = 0; i < 5; i++) {
```

```
            if (i != 4) {
```

```
                System.out.println(i + ",");
```

```
            } else {
```

```
                System.out.println(i);
```

```
            }
```

```
        }
```

```
        VariableScopeInBlock();
```

```
    }
```

```
    //语句块中变量的作用域
```

```
    private static void VariableScopeInBlock() {...}
```

```
}
```

语句块一

语句块二

语句块三

语句块中可以再套语句块，由此构成一个多层嵌套的程序结构。

从语句块的角度看程序，可以把它想象成一棵树.....



# 程序语句执行的三种方式

## 顺序执行

先执行第一句，再执行第二句，  
顺次执行到最后一句.....

## 分支（条件）结构

某些条件满足时，执行某些语句，  
不满足时，执行另外的语句

## 循环结构

某些条件满足时，反复多次地执  
行某些语句。

# 条件语句

IfStatement.java

```
//考试通过了吗?  
private static void areYouPassed() {  
    Random ran = new Random();  
    //生成一个[0,100)的随机数  
    int score = ran.nextInt( bound: 100);  
    if (score >= 60) {  
        System.out.println("恭喜恭喜,你考试及格了!");  
    } else {  
        System.out.println("杯具了,没过关!");  
    }  
}
```

使用if关键字来定义条件语句。

条件使用一个返回bool值的表达式来表达。

条件满足时,执行后面紧跟着的语句块,否则,执行else语句块。



# 条件的组合

可以使用“&&”、“||”和“!”，将多个返回bool值的表达式组合起来，构造出复杂的条件表达式。

```
private static void AndOrNot() {  
    Random ran = new Random();  
    //生成一个[0,100)的随机数  
    int ranValue = ran.nextInt( bound: 100);  
    System.out.println("ranValue=" + ranValue);  
    //【或】两个条件，只需要满足一个  
    if (ranValue % 3 == 0 || ranValue % 5 == 0) {  
        System.out.println("这个数是3或5的倍数");  
    }  
    //【与】两个条件，必须同时满足  
    if (ranValue % 2 == 0 && ranValue > 10) {  
        System.out.println("这个数是大于10的偶数");  
    }  
    //【非】你是白，我就是黑；你是黑，我就是白！  
    if (!(ranValue % 2 == 0)) {  
        System.out.println("这个数是奇数");  
    }  
}
```

# 嵌套的条件语句



右侧嵌套的条件语句，层次太深，你能使用“&&”组合法，将这个嵌套很深的条件语句，展平为只有一层吗？



```
private static void whatIsYourScore(){
    Random ran = new Random();
    //生成一个[0,100)的随机数
    int score = ran.nextInt( bound: 100);
    System.out.println("你Java考了" + score + "分");
    if(score<60){
        System.out.println("杯具了，没过关！");
    }else {
        if(score<70){
            System.out.println("好险好险，勉强过关");
        }else{
            if(score<80){
                System.out.println("考得还算过得去，中不溜");
            }else{
                if(score<90){
                    System.out.println("考得不错，良！");
                }else{
                    System.out.println("你是学霸！");
                }
            }
        }
    }
}
```

1

2

3

4



# "?:"表达式

语法格式:

返回true或false值  
的表达式

?

前值=true时, 返回的值

:

前值=false时, 返回的值

示例:

```
int score = 61;  
String result = "";  
if (score > 60) {  
    result = "过了!";  
} else {  
    result = "挂了!";  
}  
System.out.println(result);
```

=

```
int score = 61;  
String result = score > 60 ? "过了!" : "挂了!";  
//输出: 过了!  
System.out.println(result);
```



"?:", 其实就是一个精简的条件语句。

# 多分支语句

嵌套多层的条件语句，往往可以使用switch语句进行简化。



```
private static void useSwitchStatement() {  
    Random ran = new Random();  
    //生成一个[0,100)的随机数  
    int score = ran.nextInt( bound: 100);  
    System.out.println("你Java考了" + score + "分");  
    //成绩除以10，以便判断此成绩归于哪个成绩组  
    int result=score/10;  
    switch (result) {  
        case 0, 1, 2, 4, 5:  
            System.out.println("杯具了，没过关!");  
            break;  
        case 6:  
            System.out.println("好险好险，勉强过关");  
            break;  
        case 7:  
            System.out.println("考得还算过得去，中不溜");  
            break;  
        case 8:  
            System.out.println("考得不错，良!");  
            break;  
        default:  
            System.out.println("你是学霸!");  
    }  
}
```

SwitchStatement.java

# switch表达式

```
private static void useSwitchExpression() {  
    Random ran = new Random();  
    //生成一个[0,100)的随机数  
    int score = ran.nextInt( bound: 100);  
    System.out.println("你Java考了" + score + "分");  
    //成绩除以10，以便判断此成绩归于哪个成绩组  
    int flag = score / 10;  
    //switch表达式，会返回一个值作为结果，因此，一定要有default分支  
    String result = switch (flag) {  
        case 0, 1, 2, 4, 5 -> "杯具了，没过关！";  
        case 6 -> "好险好险，勉强过关";  
        case 7 -> "考得还算过得去，中不溜";  
        case 8 -> "考得不错，良！";  
        default -> "你是学霸！";  
    };  
    System.out.println(result);  
}
```

这是JDK 14 引入的特性。

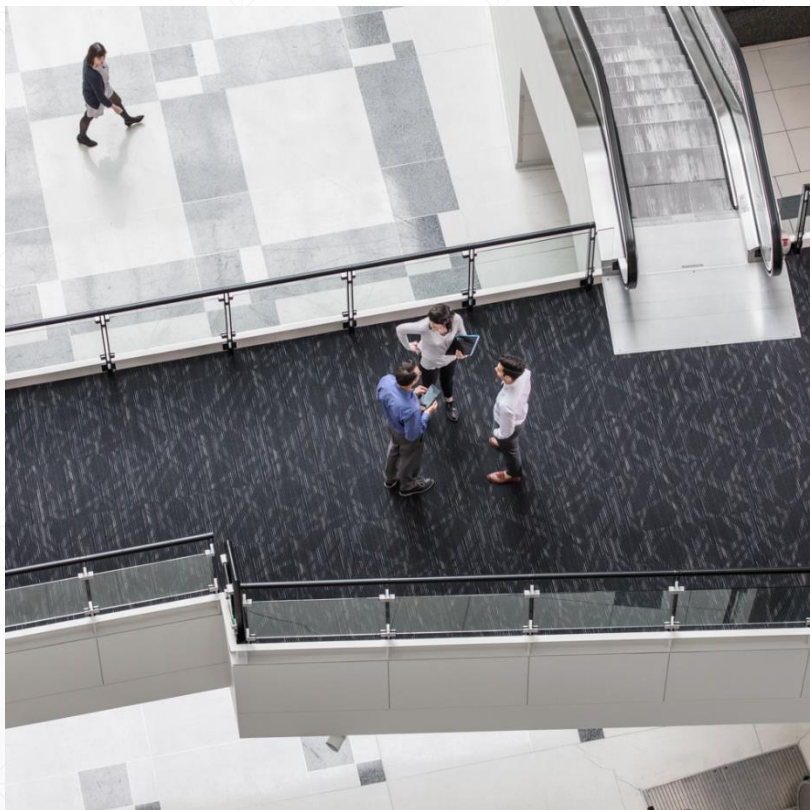
整个switch结构被当成是一个表达式，它一定返回有一个值，它的分支中一定有一个default分支，以保证能覆盖所有可能的情况。

# 巩固练习



编写一个程序，用户输入一个数字，比如“7”，程序输出相应的英文月份名称，比如“July”。

# 循环结构-1



先做事，再谈条件

```
//使用do/while循环输出: 1,2,3,4,5,6,7,8,9,10
private static void DoWhileLoop() {
    int i = 1;
    do {
        if (i != 10)
            System.out.print(i + ",");
        else
            System.out.println(i);
        i++;
    } while (i <= 10);
}
```

LoopStatement.java

# 循环结构-2



先谈条件，再做事

```
//使用do/while循环输出: 1,2,3,4,5,6,7,8,9,10
private static void whileDoLoop() {
    int i = 1;
    while (i<=10){
        if (i != 10)
            System.out.print(i + ",");
        else
            System.out.println(i);
        i++;
    };
}
```



## 循环结构-3

当循环执行的次数已知时，使用for循环最合适：

```
//使用for循环输出：1,2,3,4,5,6,7,8,9,10
private static void forLoop(){
    for(int i=1;i<=10;i++){
        if (i != 10)
            System.out.print(i + ",");
        else
            System.out.println(i);
    }
}
```



# 学会使用for循环



1

2

4

```
for(循环变量初始值; 循环必须满足的条件; 循环最后执行的操作){  
    //循环体语句块  
}
```

3

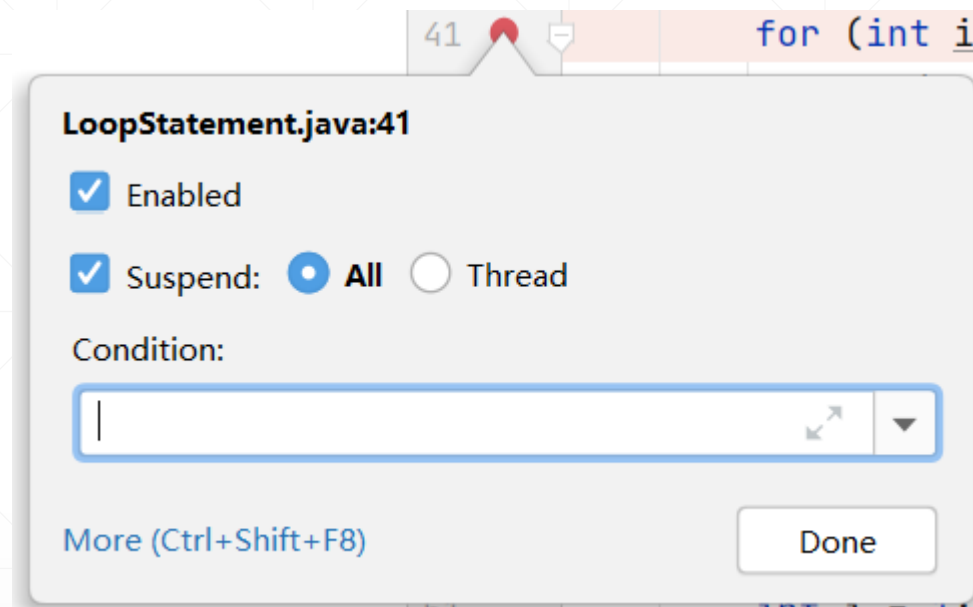
循环开始时，先执行第①步，给循环变量一个初始值  
然后，判断②所放置的表达式其值是不是为true，为true时执行放在③处的语句块，执行完毕之后，再执行第④个表达式，通常它会修改循环变量的值。

只要②所放置的表达式其值保持为true，放在③处的语句块就会被反复执行。  
当表达式的值变为false时，循环中止。

# 学会使用IntelliJ的调试功能

在代码左侧空白区域点击，可以给代码打上断点。

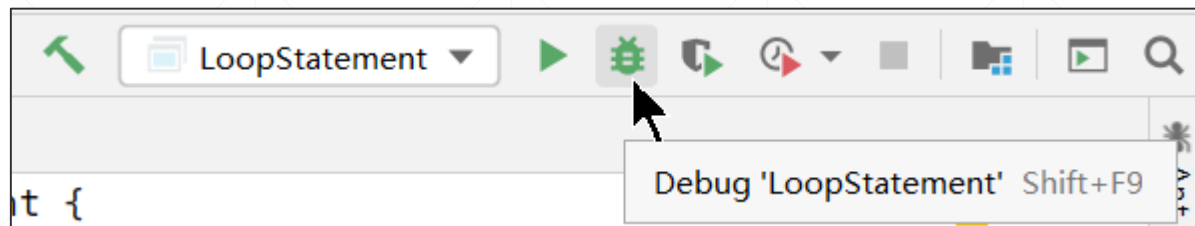
```
39      //使用for循环输出: 1,2,3,4,5,6,7,8,9,10
40      private static void forLoop() {
41          for (int i = 1; i <= 10; i++) {
42              if (i != 10)
43                  System.out.print(i + ",");
44              else
45                  System.out.println(i);
46          }
47      }
```



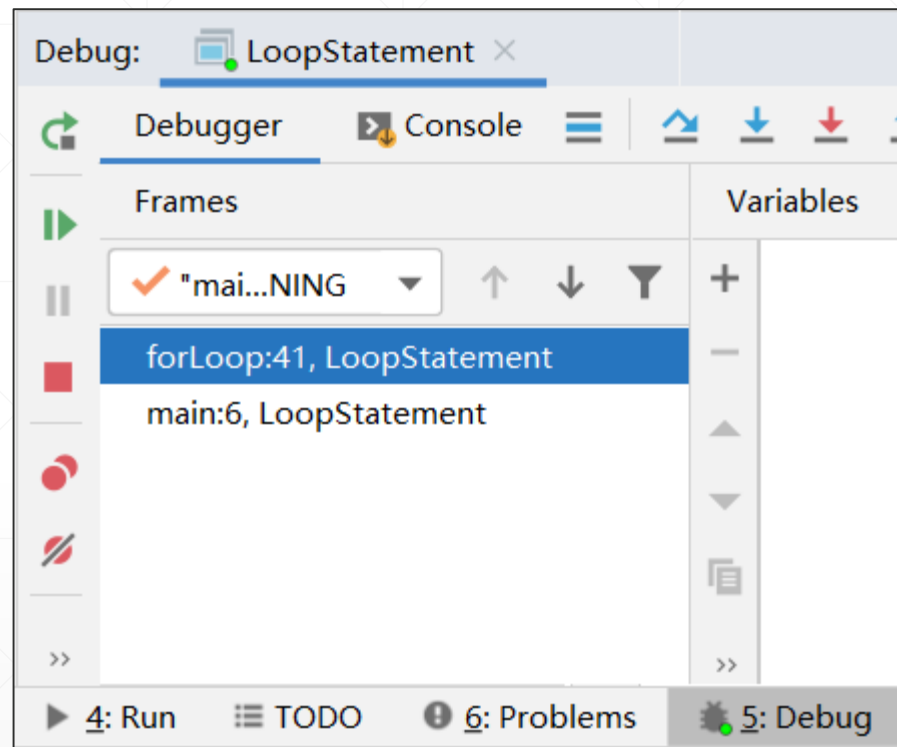
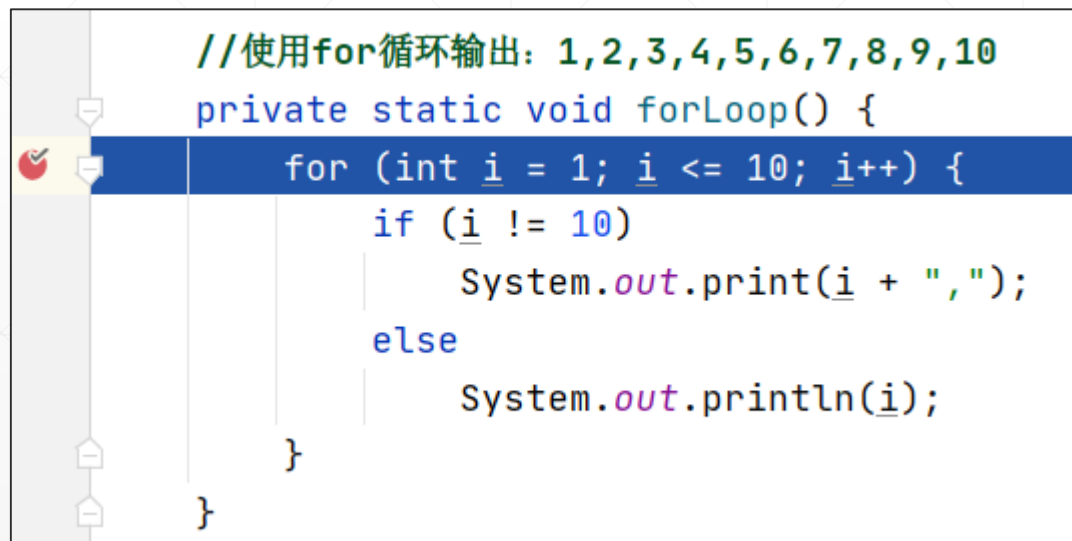
在小红点上右击，可以设置在什么情况下启用或禁用这个断点。

# 启动调试

点击工具栏上的“小虫”图标启动调试进程：



程序会在断点处“暂停”：



Debug面板出现，它提供了许多功能用于调试程序

单步执行（不  
进入函数内部）

单步执行（进入  
函数内部）

快速执行当前函数后继代  
码（跳出函数）

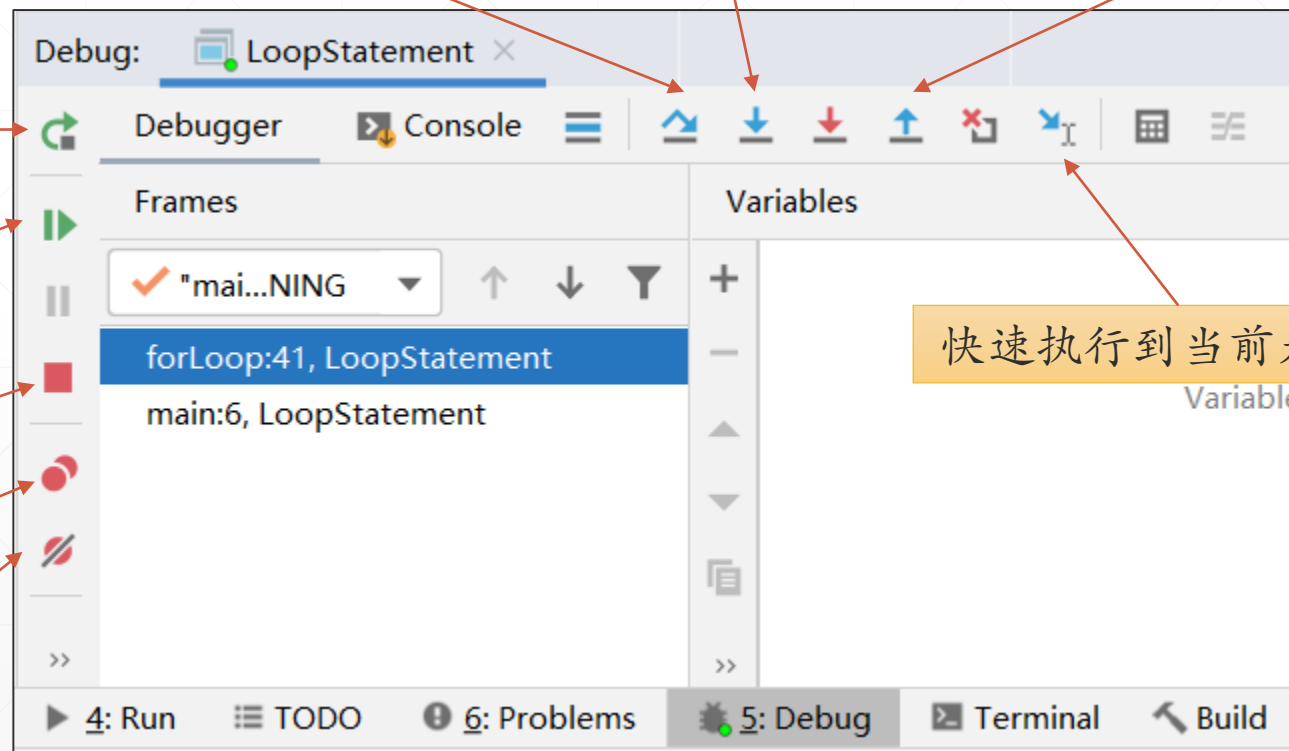
重新运行程序

继续运行程序（到下  
一个断点）

停止程序调试

查看程序断点信息

暂时禁用断点

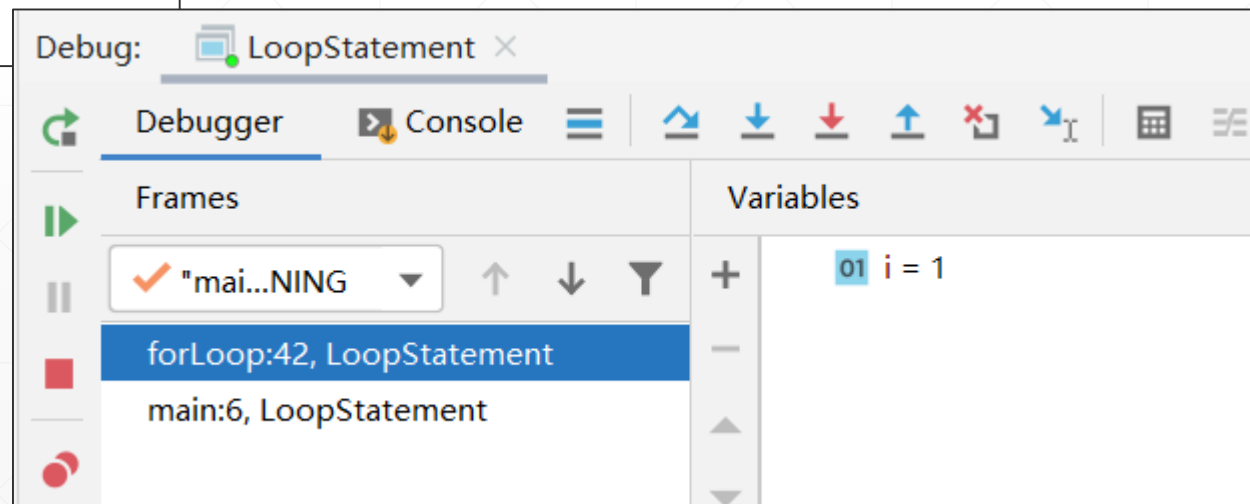


# 查看局部变量的值

```
//使用for循环输出: 1,2,3,4,5,6,7,8,9,10
private static void forLoop() {
    for (int i = 1; i <= 10; i++) { i: 1
        if (i != 10 = true) i: 1
            System.out.print(i + ",");
        else
            System.out.print + "1,"
    }
}
```

Debug面板中有一个Variables子面板，也能查看变量的当前值

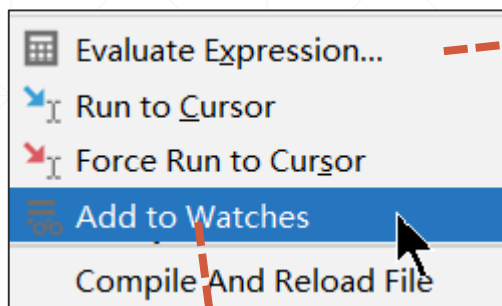
鼠标移到变量附近，可以直接查看变量的当前值。



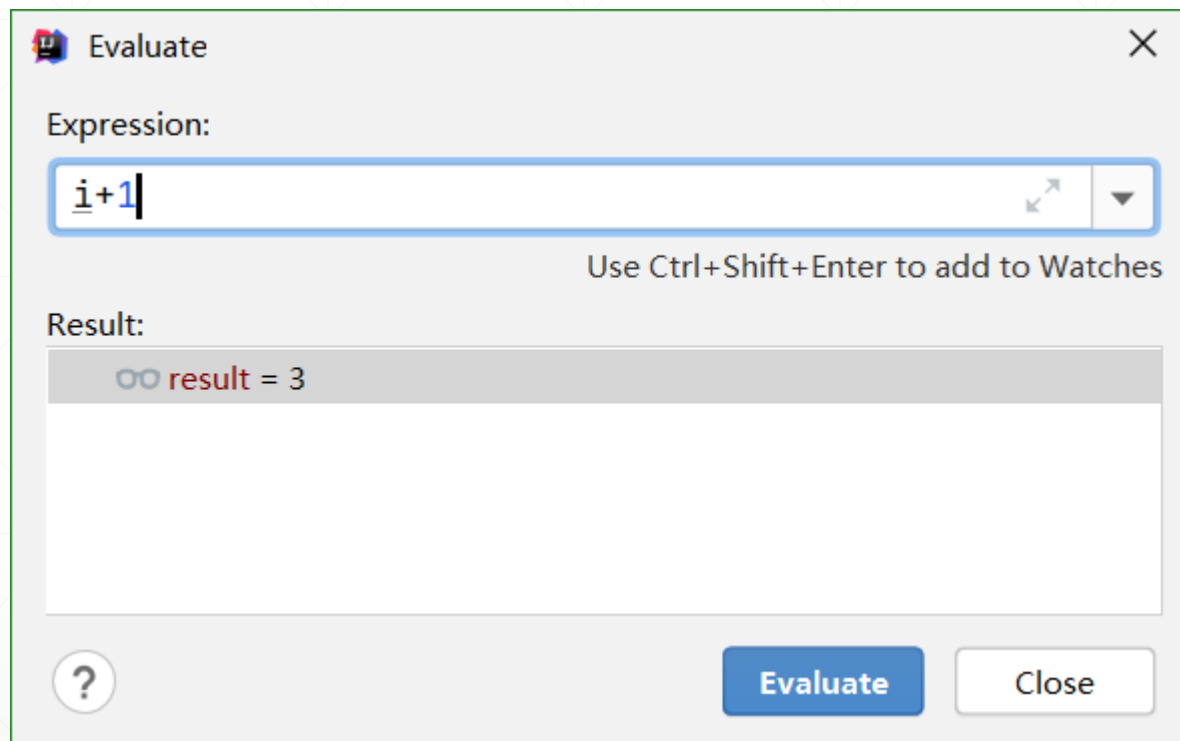
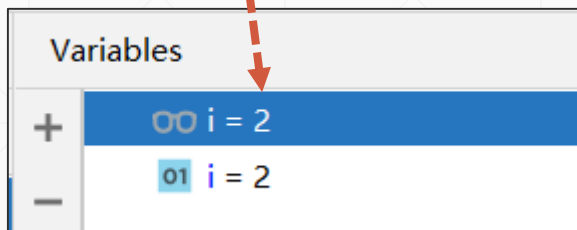


# 监视变量值的变化与评估表达式的值

在变量单词上右击，可以找到以下与调试相关的命令：

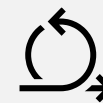


添加对变量*i*的监视：



使用Evaluate窗口，可以使用当前变量进行一些简单的运算，或查看特定表达式在当前状态下的值。

# 死循环



在某些情况下，你可能不知道到底需要执行多少次循环，这时，可以采用“死循环” + “break” 实现：

//使用DoWhile实现的“死循环示例”

```
private static void infiniteDoWhileLoop() {  
    int i = 1;  
    do {  
        if (i != 10)  
            System.out.print(i + ",");  
        else  
            System.out.println(i);  
        i++;  
        if (i > 10) break; //终止循环  
    } while (true);  
}
```

//使用For实现的“死循环示例”

```
private static void infiniteForLoop() {  
    for (int i = 1; ; i++) {  
        if (i > 10) break; //终止循环  
        if (i != 10)  
            System.out.print(i + ",");  
        else  
            System.out.println(i);  
    }  
}
```



break用于提前结束整个循环。

# 使用continue跳过本轮循环，执行下一轮循环

```
//使用continue跳过偶数，只输出奇数:1,3,5,7,9,  
private static void continueInLoop() {  
    for (int i = 1; i <= 10; i++) {  
        //如果是偶数，提前结束本轮循环  
        if (i % 2 == 0) continue;  
        System.out.print(i + ",");  
    }  
}
```

continue的功能与break不一样，请仔细阅读示例代码，分清楚其差别。

# 多学一点：在线查看Unicode字符集

<https://unicode-table.com/>

```
char c = '\u03B6';  
//输出: ζ  
System.out.println(c);
```

|      | 0      | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | A     | B     | C     | D     | E     | F     |
|------|--------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|
| 0000 | [NUL]  | [SOH] | [STX] | [ETX] | [EOT] | [ENQ] | [ACK] | [BEL] | [BS]  | [HT]  | [LF]  | [VT]  | [FF]  | [CR]  | [SO]  | [SI]  |
| 0010 | [DLE]  | [DC1] | [DC2] | [DC3] | [DC4] | [NAK] | [SYN] | [ETB] | [CAN] | [EM]  | [SUB] | [ESC] | [FS]  | [GS]  | [RS]  | [US]  |
| 0020 |        | !     | "     | #     | \$    | %     | &     | '     | (     | )     | *     | +     | ,     | -     | .     | /     |
| 0030 | 0      | 1     | 2     | 3     | 4     | 5     | 6     | 7     | 8     | 9     | :     | ;     | <     | =     | >     | ?     |
| 0040 | @      | A     | B     | C     | D     | E     | F     | G     | H     | I     | J     | K     | L     | M     | N     | O     |
| 0050 | P      | Q     | R     | S     | T     | U     | V     | W     | X     | Y     | Z     | [     | \     | ]     | ^     | _     |
| 0060 | `      | a     | b     | c     | d     | e     | f     | g     | h     | i     | j     | k     | l     | m     | n     | o     |
| 0070 | p      | q     | r     | s     | t     | u     | v     | w     | x     | y     | z     | {     |       | }     | ~     | ¸     |
| 0080 | [XXX]  | [XXX] | [BPH] | [NBH] | [IND] | [NEL] | [SSA] | [ESA] | [HTS] | [HTJ] | [VTS] | [PLD] | [PLU] | [RI]  | [SK2] | [SS3] |
| 0090 | [dCS]  | [PU1] | [PU2] | [STS] | [CCH] | [MW]  | [SPA] | [ERA] | [SOS] | [XXX] | [SD1] | [CSI] | [ST]  | [OSC] | [PM]  | [APC] |
| 00A0 | [NBSP] | ı     | ç     | £     | ¤     | ¥     | ¦     | §     | ¨     | ©     | ª     | «     | ¬     |       | ®     | ¯     |
| 00B0 | °      | ±     | ²     | ³     | ´     | µ     | ¶     | ·     | ¸     | ¹     | º     | »     | ¼     | ½     | ¾     | ¿     |
| 00C0 | À      | Á     | Â     | Ã     | Ä     | Å     | Æ     | Ç     | È     | É     | Ê     | Ë     | Ì     | Í     | Î     | Ï     |
| 00D0 | Ð      | Ñ     | Ò     | Ó     | Ô     | Õ     | Ö     | ×     | Ø     | Ù     | Ú     | Û     | Ü     | Ý     | Þ     | ß     |
| 00E0 | à      | á     | â     | ã     | ä     | å     | æ     | ç     | è     | é     | ê     | ë     | ì     | í     | î     | ï     |

## Latin Extended-A ▾

[Open in an individual page](#)

Range: [0100–017F](#)

Click to highlight range

Languages: celtic, sami, maltese, turkish

# 编程练习



编写一个程序，完成以下功能：

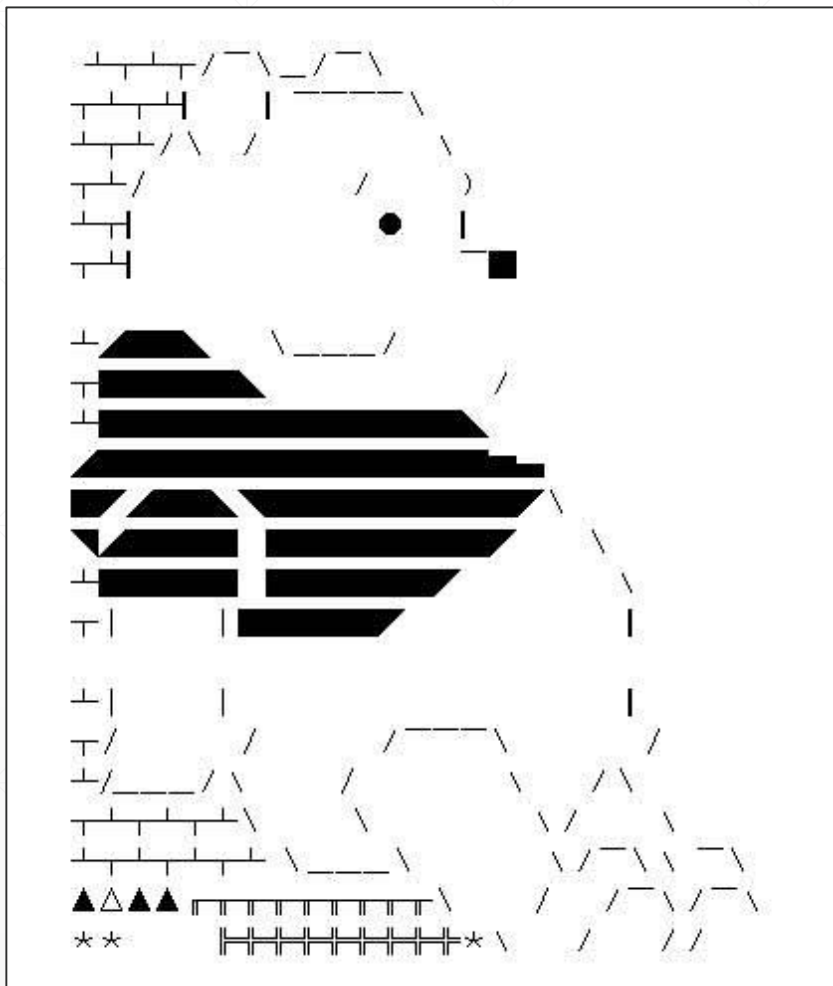
- (1) 使用循环语句，输出Unicode字符集中的所有字符。
- (2) 让用户输入特定字符的Unicode数值，程序输出这个符号。

提示：

char类型的变量，可以直接接收一个整数值。

你可以将Unicode字符对应的“十六进制”数值，转换为十进制数值。

# 留给学霸的技术探险任务



网上有一些人使用字符拼出了许多有趣的图片，甚至还有人写出了相应程序，能将任意一张图片转换为“字符图”。

上网搜集一下相关资料，了解一下相关原理，然后，思索一下，我以后能不能用Java也写一个这样的“字符图形”生成器呢？这个可以当成一个中长期的学习目标。

下一讲，我们介绍如何将语句组合起来，设计和编写  
“方法 (method)” .....